LAB 9

The Movie Manager App helps users track and manage their personal movie collection. The app allows users to perform basic CRUD operations (Create, Read, Update, Delete) on movie entries. All data must be stored locally using **SQLite** and the app must run on both Android and iOS via **Expo**.

Data Structure

Each movie includes the following attributes:

id *integer* Unique identifier (auto-incremented by SQLite)
movie_title *text* Title of the movie (e.g., "Inception")
genre *text* Genre of the movie (e.g., "Sci-Fi", "Action", "Drama")
release_year *integer* Release year of the movie (e.g., 2010)
rating *real* User rating from 0.0 to 5.0 (e.g., 4.5)

General Requirements:

- Technology: **EXPO** to build the application
- Data Storage: Use **SQLite**
- Navigation: Use **Bottom Tab Navigator** to switch between screens
- UI: Clean and functional interface with proper styling

Task 1: Display Movies List

Create the main screen that displays all movies from the database:

- Display movies in a **FlatList** showing all movie attributes
- Show a header with app title "Movie Manager"
- Implement pull-to-refresh functionality to reload data

Task 2: Add New Movie

Create a screen to add new movies to the collection:

- Form with input fields for: Movie Title, Genre, Release Year, Rating
- Save button that stores the new movie in **SQLite**
- Success message after adding and automatic navigation back to movie list

Task 3: Movie Detail View with Actions

Create a movie detail screen that displays complete movie information and provides actions:

- When tapping a movie in the list, navigate to detail screen showing all movie attributes (1.0 point)
- Display all movie information in a readable layout
- Include two action buttons:
    - **Edit Button**: Navigates to edit screen with pre-filled data, Edit Button change label to Save label.
    - **Delete Button**: Shows confirmation dialog and deletes movie from database

Task 4: Search Movies

Implement search functionality to find movies:

- Add search bar at the top of movies list screen
- Real-time filtering of movies by title
- Display search results in the same FlatList

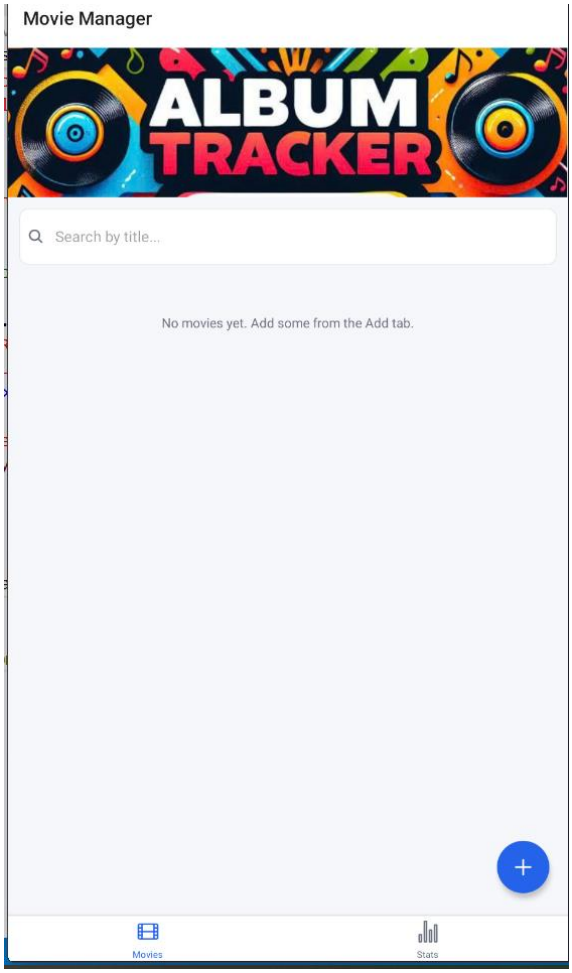
Task 5: Calculate Total Statistics

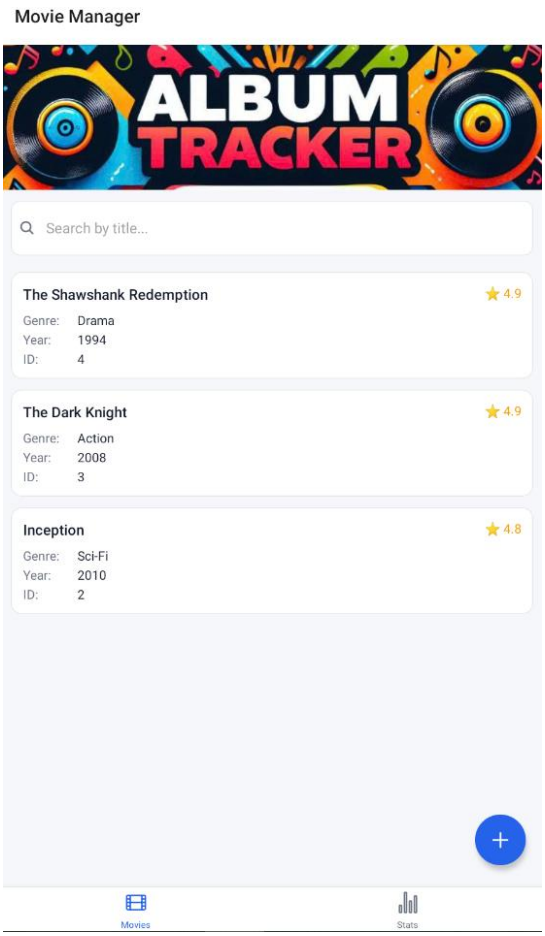
Create a statistics screen that shows:

- Total number of movies in collection
- Average rating of all movies
- Movies count by genre

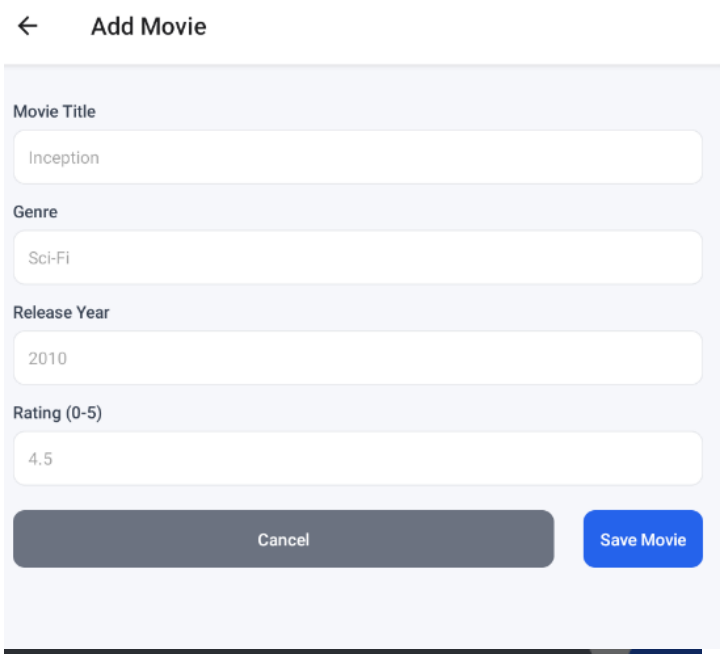
PhucPT@fpt.edu.vn

ID	Movie Title	Genre	Release Year	Rating
1	Inception	Sci-Fi	2010	4.8
2	The Dark Knight	Action	2008	4.9
3	The Shawshank Redemption	Drama	1994	4.9
4	Pulp Fiction	Crime	1994	4.7
5	Forrest Gump	Drama	1994	4.5
6	The Matrix	Sci-Fi	1999	4.6
7	The Godfather	Crime	1972	4.9
8	Avatar	Adventure	2009	4.3
9	Interstellar	Sci-Fi	2014	4.7
10	Parasite	Thriller	2019	4.8

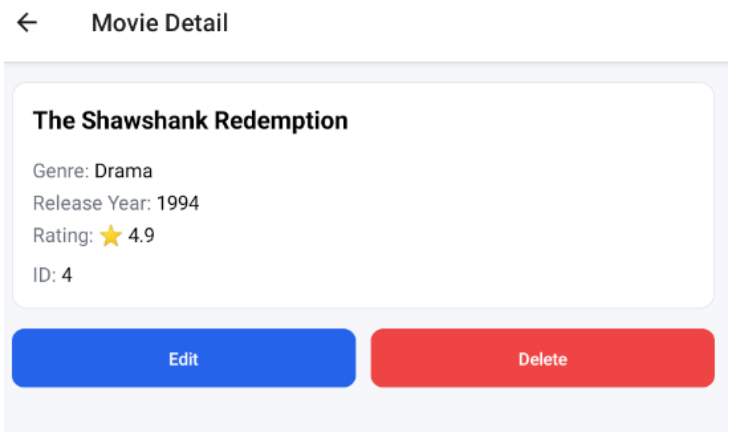




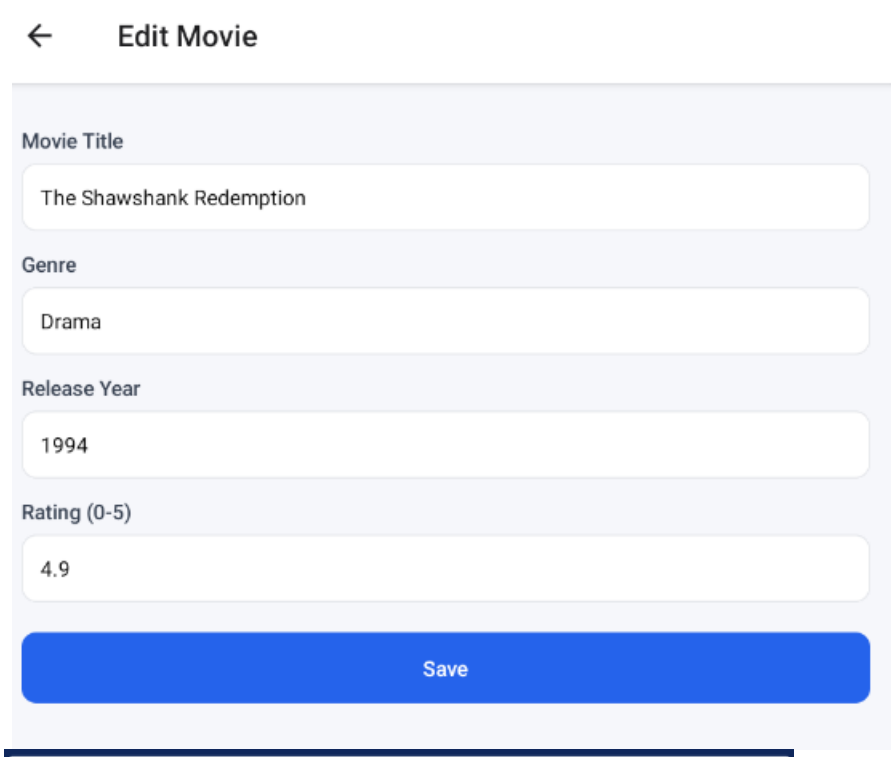
PhucPT@fpt.edu.vn

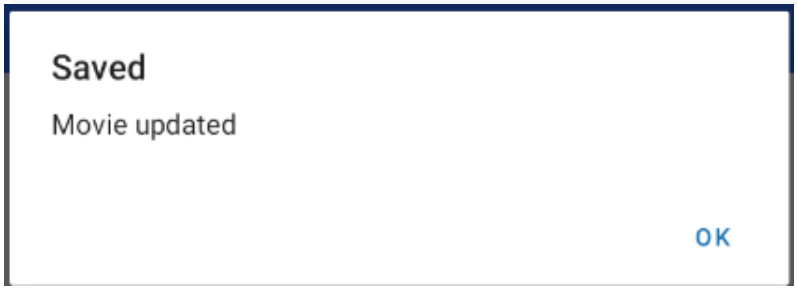




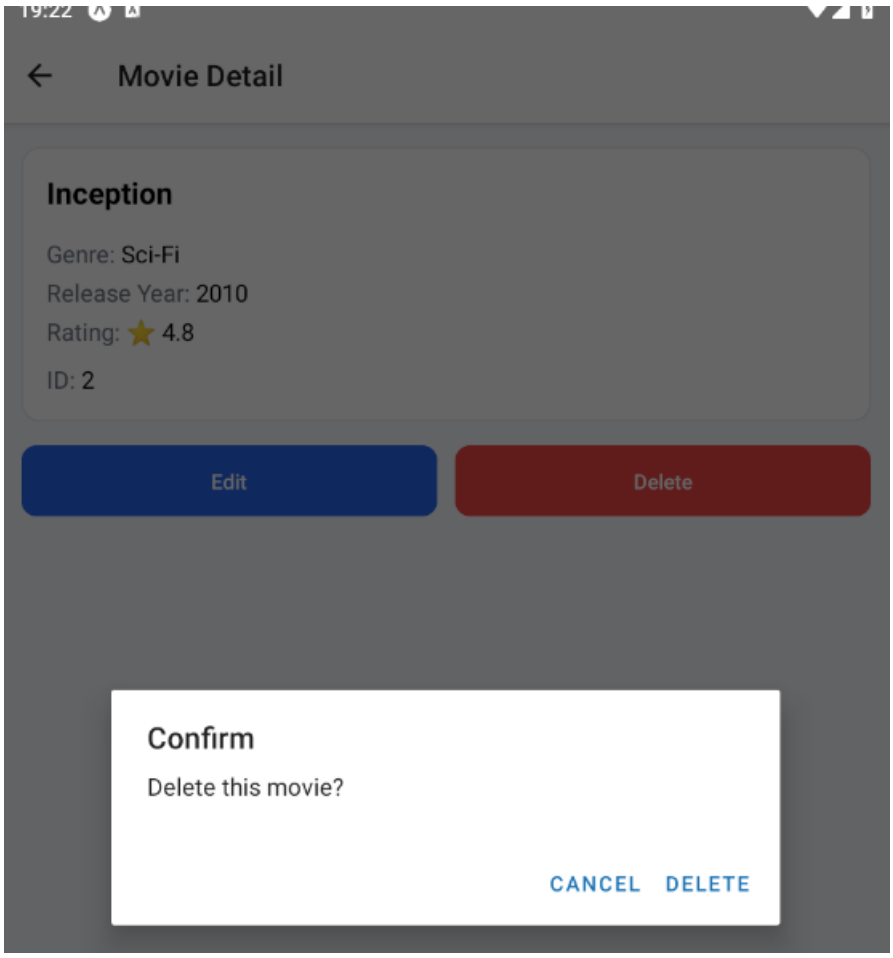


PhucPT@fpt.edu.vn

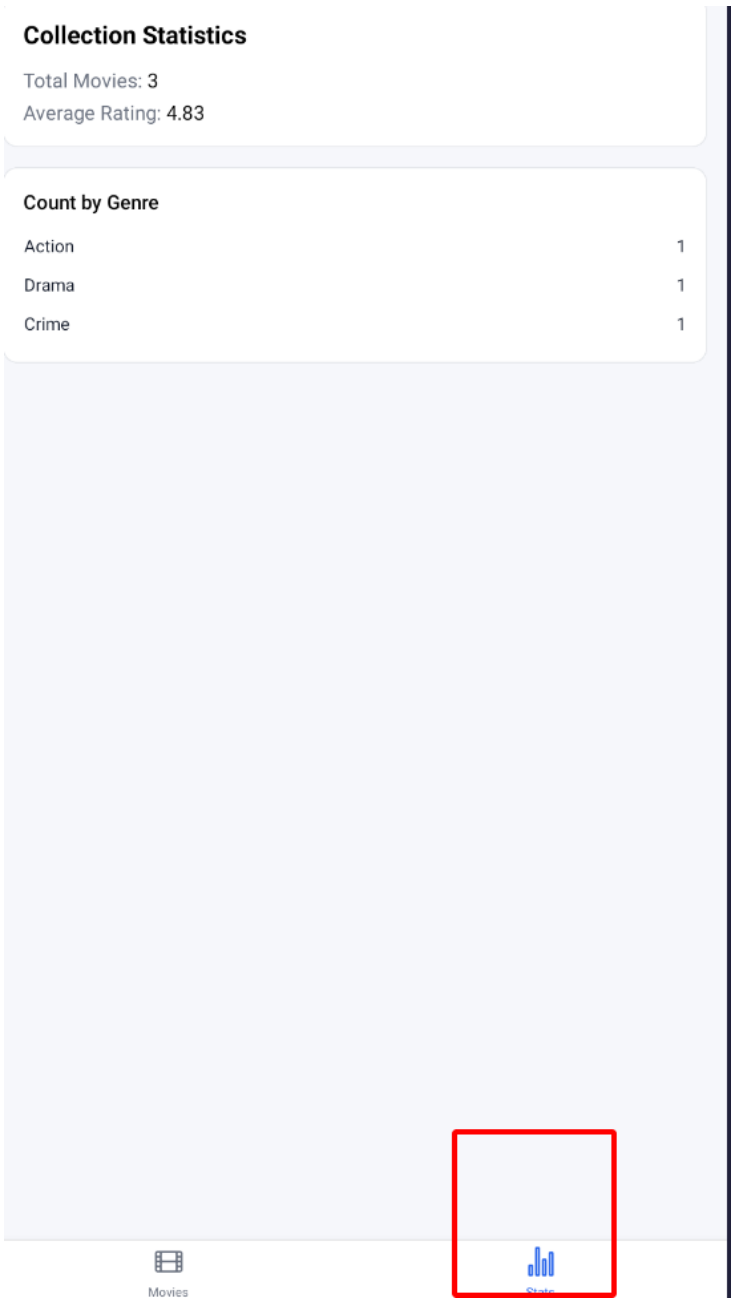




PhucPT@fpt.edu.vn

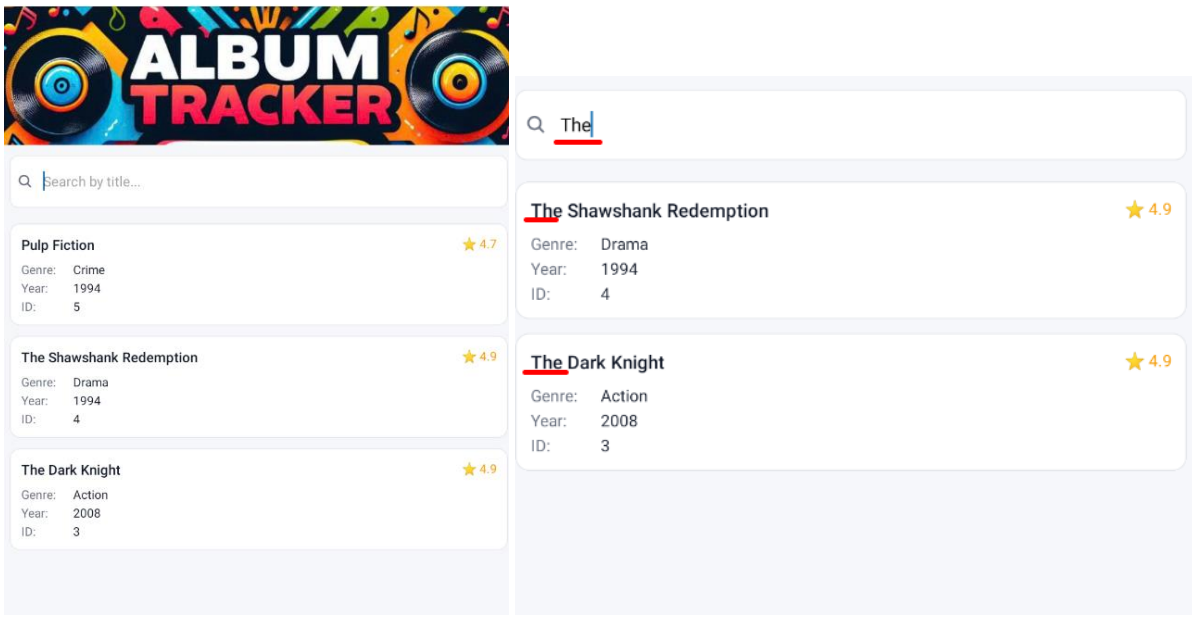


PhucPT@fpt.edu.vn



PhucPT@fpt.edu.vn

Movie Manager



PhucPT@fpt.edu.vn

Tóm tắt nhanh (summary)

1. Khởi tạo project Expo (nếu bạn chưa có): `npx create-expo-app moviesqlite` hoặc dùng template tương tự.
2. Cài dependencies (theo `package.json`): @react-navigation, expo-sqlite, v.v. (mình sẽ chỉ lệnh cụ thể).
3. • Tạo cấu trúc thư mục và các file: `App.js`, `index.js`, `database.js`, `screens/*` (MoviesList, MovieDetail, AddMovie, EditMovie, Statistics).
4. • Viết mã: database helper (SQLite), stack & tab navigation, các màn hình CRUD, các component UI cơ bản.
5. • Chạy app bằng Expo, kiểm tra chức năng: thêm, sửa, xóa phim; xem chi tiết; thống kê.
6. • Debugging & kiểm tra: log, console, kiểm tra transaction SQL, permission (nếu cần).

Công nghệ & phụ thuộc chính

- Expo (SDK ~54) — xem `package.json`.
- React Native (0.81.x) và React 19.x.
- Navigation: `@react-navigation/native`, `@react-navigation/native-stack`, `@react-navigation/bottom-tabs`.
- SQLite: `expo-sqlite`.
- Gesture handler / Reanimated / Safe area / Screens (bắt buộc cho navigation).

Cấu trúc dự án (đề xuất)

```
moviesqlite/
├─ App.js
├─ index.js
├─ package.json
├─ babel.config.js
├─ database.js
├─ screens/
│  ├─ MoviesListScreen.js
│  ├─ MovieDetailScreen.js
│  ├─ AddMovieScreen.js
│  ├─ EditMovieScreen.js
│  └─ StatisticsScreen.js
├─ components/
│  └─ MovieItem.js   (tùy chọn)
└─ assets/
   └─ (hình, icons nếu cần)
```

Hướng dẫn từng bước (chi tiết, có mã mẫu)

1) Tạo project (nếu bắt đầu từ đầu)
   npx create-expo-app moviesqlite –template blank

PhucPT@fpt.edu.vn

cd moviesqlite
npm install @react-navigation/native @react-navigation/native-stack @react-navigation/bottom-tabs
npm install react-native-gesture-handler react-native-reanimated react-native-safe-area-context react-native-screens
npm install expo-sqlite

Nếu dùng Expo Managed workflow, hầu hết packages hoạt động ngay. Với `react-native-reanimated` cần cấu hình theo doc nếu gặp lỗi (bật plugin babel).

2) Cấu hình babel (nếu dùng reanimated)

```
module.exports = function(api) {
 api.cache(true);
 return {
  presets: ['babel-preset-expo'],
  plugins: [
   'react-native-reanimated/plugin' // phải là plugin cuối cùng
  ],
 };
};
```

(Đặt plugin reanimated ở cuối danh sách plugin.)

3) Tạo file `database.js` — helper cho SQLite

Mục đích: mở database, tạo table (movies), expose hàm CRUD.

database.js:

```
import * as SQLite from 'expo-sqlite';

const db = SQLite.openDatabaseSync('movies.db');

// Initialize database
export const initDatabase = () => {
    try {
        db.execSync(`
      CREATE TABLE IF NOT EXISTS movies (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        movie_title TEXT NOT NULL,
        genre TEXT NOT NULL,
        release_year INTEGER NOT NULL,
        rating REAL NOT NULL
      );
    `);
        console.log('Database initialized successfully');
    } catch (error) {
        console.error('Error initializing database:', error);
    }
};

// Insert sample data
```


PhucPT@fpt.edu.vn

```
export const insertSampleData = () => {
    try {
        const count = db.getFirstSync('SELECT COUNT(*) as count FROM movies');

        if (count.count === 0) {
            const sampleMovies = [
                { title: 'Inception', genre: 'Sci-Fi', year: 2010, rating: 4.8
},
                { title: 'The Dark Knight', genre: 'Action', year: 2008, rating:
4.9 },
                { title: 'The Shawshank Redemption', genre: 'Drama', year: 1994,
rating: 4.9 },
                { title: 'Pulp Fiction', genre: 'Crime', year: 1994, rating: 4.7
},
                { title: 'Forrest Gump', genre: 'Drama', year: 1994, rating: 4.5
},
                { title: 'The Matrix', genre: 'Sci-Fi', year: 1999, rating: 4.6
},
                { title: 'The Godfather', genre: 'Crime', year: 1972, rating:
4.9 },
                { title: 'Avatar', genre: 'Adventure', year: 2009, rating: 4.3
},
                { title: 'Interstellar', genre: 'Sci-Fi', year: 2014, rating:
4.7 },
                { title: 'Parasite', genre: 'Thriller', year: 2019, rating: 4.8
}
            ];

            sampleMovies.forEach(movie => {
                db.runSync(
                    'INSERT INTO movies (movie_title, genre, release_year,
rating) VALUES (?, ?, ?, ?)',
                    [movie.title, movie.genre, movie.year, movie.rating]
                );
            });
            console.log('Sample data inserted successfully');
        }
    } catch (error) {
        console.error('Error inserting sample data:', error);
    }
};

// Get all movies
export const getAllMovies = () => {
    try {
        const movies = db.getAllSync('SELECT * FROM movies ORDER BY id DESC');
        return movies;
    } catch (error) {
        console.error('Error getting movies:', error);
        return [];
    }
};

// Search movies by title
export const searchMovies = (searchText) => {
    try {
        const movies = db.getAllSync(
            'SELECT * FROM movies WHERE movie_title LIKE ? ORDER BY id DESC',
```

```
            [`%${searchText}%`]
        );
        return movies;
    } catch (error) {
        console.error('Error searching movies:', error);
        return [];
    }
};

// Get movie by ID
export const getMovieById = (id) => {
    try {
        const movie = db.getFirstSync('SELECT * FROM movies WHERE id = ?',
[id]);
        return movie;
    } catch (error) {
        console.error('Error getting movie by ID:', error);
        return null;
    }
};

// Add new movie
export const addMovie = (movieTitle, genre, releaseYear, rating) => {
    try {
        const result = db.runSync(
            'INSERT INTO movies (movie_title, genre, release_year, rating)
VALUES (?, ?, ?, ?)',
            [movieTitle, genre, releaseYear, rating]
        );
        return result.lastInsertRowId;
    } catch (error) {
        console.error('Error adding movie:', error);
        return null;
    }
};

// Update movie
export const updateMovie = (id, movieTitle, genre, releaseYear, rating) => {
    try {
        db.runSync(
            'UPDATE movies SET movie_title = ?, genre = ?, release_year = ?,
rating = ? WHERE id = ?',
            [movieTitle, genre, releaseYear, rating, id]
        );
        return true;
    } catch (error) {
        console.error('Error updating movie:', error);
        return false;
    }
};

// Delete movie
export const deleteMovie = (id) => {
    try {
        db.runSync('DELETE FROM movies WHERE id = ?', [id]);
        return true;
    } catch (error) {
        console.error('Error deleting movie:', error);
```

```
        return false;
    }
};

// Get statistics
export const getStatistics = () => {
    try {
        const totalMovies = db.getFirstSync('SELECT COUNT(*) as count FROM
movies');
        const avgRating = db.getFirstSync('SELECT AVG(rating) as average FROM
movies');
        const genreStats = db.getAllSync(
            'SELECT genre, COUNT(*) as count FROM movies GROUP BY genre ORDER BY
count DESC'
        );

        return {
            totalMovies: totalMovies.count,
            averageRating: avgRating.average ? avgRating.average.toFixed(2) : 0,
            genreStats: genreStats
        };
    } catch (error) {
        console.error('Error getting statistics:', error);
        return {
            totalMovies: 0,
            averageRating: 0,
            genreStats: []
        };
    }
};

export default db;
```

Giải thích:

- `initDB()` cần gọi khi app khởi động để đảm bảo table tồn tại.
- Mỗi hàm trả Promise để dễ dùng `async/await` trong components.

4) `App.js` — khởi tạo Navigation và khởi tạo DB

- Mục đích: setup Navigation (Tab + Stack), gọi `initDB()` khi app mount.

`App.js`:

```
import React, { useEffect } from 'react';
import { StatusBar } from 'expo-status-bar';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { Ionicons } from '@expo/vector-icons';
import { initDatabase } from './src/database/database';
```

```
// Import screens
import MoviesListScreen from './src/screens/MoviesListScreen';
import AddMovieScreen from './src/screens/AddMovieScreen';
import MovieDetailScreen from './src/screens/MovieDetailScreen';
import EditMovieScreen from './src/screens/EditMovieScreen';
import StatisticsScreen from './src/screens/StatisticsScreen';

const Tab = createBottomTabNavigator();
const Stack = createNativeStackNavigator();

// Movies Stack Navigator
function MoviesStack() {
  return (
    <Stack.Navigator
      screenOptions={{
        headerShown: false,
      }}
    >
      <Stack.Screen name="MoviesList" component={MoviesListScreen} />
      <Stack.Screen
        name="Add Movie"
        component={AddMovieScreen}
        options={{
          headerShown: true,
          headerTitle: 'Add New Movie',
          headerStyle: {
            backgroundColor: '#6200ee',
          },
          headerTintColor: '#fff',
          headerTitleStyle: {
            fontWeight: 'bold',
          },
        }}
      />
      <Stack.Screen
        name="MovieDetail"
        component={MovieDetailScreen}
        options={{
          headerShown: true,
          headerTitle: 'Movie Details',
          headerStyle: {
            backgroundColor: '#6200ee',
          },
          headerTintColor: '#fff',
          headerTitleStyle: {
            fontWeight: 'bold',
          },
        }}
      />
      <Stack.Screen
```


PhucPT@fpt.edu.vn

```
        name="EditMovie"
        component={EditMovieScreen}
        options={{
          headerShown: true,
          headerTitle: 'Edit Movie',
          headerStyle: {
            backgroundColor: '#2196F3',
          },
          headerTintColor: '#fff',
          headerTitleStyle: {
            fontWeight: 'bold',
          },
        }}
      />
    </Stack.Navigator>
  );
}

export default function App() {
  useEffect(() => {
    // Initialize database only (no sample data)
    initDatabase();
  }, []);

  return (
    <NavigationContainer>
      <StatusBar style="light" />
      <Tab.Navigator
        screenOptions={({ route }) => ({
          tabBarIcon: ({ focused, color, size }) => {
            let iconName;

            if (route.name === 'Movies') {
              iconName = focused ? 'film' : 'film-outline';
            } else if (route.name === 'Statistics') {
              iconName = focused ? 'stats-chart' : 'stats-chart-outline';
            }

            return <Ionicons name={iconName} size={size} color={color} />;
          },
          tabBarActiveTintColor: '#6200ee',
          tabBarInactiveTintColor: 'gray',
          tabBarStyle: {
            paddingBottom: 5,
            paddingTop: 5,
            height: 60,
          },
          tabBarLabelStyle: {
            fontSize: 12,
            fontWeight: '600',
```


PhucPT@fpt.edu.vn

```
          },
          headerShown: false,
        })}
      >
        <Tab.Screen name="Movies" component={MoviesStack} />
        <Tab.Screen name="Statistics" component={StatisticsScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

Giải thích:

- `MoviesStack` chứa các màn hình liên quan đến movie.
- Tab gồm Home (stack) và Statistics.
- `initDB()` gọi 1 lần ở `useEffect` để tạo table.

5) `MoviesListScreen.js` — danh sách phim (Read)

Mục đích: hiển thị tất cả phim từ DB, link tới add/edit/detail.

screens/MoviesListScreen.js:

```
import React, { useState, useEffect, useCallback } from 'react';
import {
    View,
    Text,
    FlatList,
    StyleSheet,
    TouchableOpacity,
    RefreshControl,
    TextInput,
    Alert,
    Image
} from 'react-native';
import { getAllMovies, searchMovies } from '../database/database';
import { Ionicons } from '@expo/vector-icons';

export default function MoviesListScreen({ navigation }) {
    const [movies, setMovies] = useState([]);
    const [refreshing, setRefreshing] = useState(false);
    const [searchText, setSearchText] = useState('');

    useEffect(() => {
        // Load movies on initial mount
        loadMovies();

        const unsubscribe = navigation.addListener('focus', () => {
```


PhucPT@fpt.edu.vn

```
            loadMovies();
        });

        return unsubscribe;
    }, [navigation]);

    const loadMovies = () => {
        try {
            const data = getAllMovies();
            setMovies(data);
        } catch (error) {
            console.error('Error loading movies:', error);
            // Retry after a short delay if database not ready
            setTimeout(() => {
                try {
                    const data = getAllMovies();
                    setMovies(data);
                } catch (retryError) {
                    console.error('Retry failed:', retryError);
                }
            }, 100);
        }
    };

    const onRefresh = useCallback(() => {
        setRefreshing(true);
        loadMovies();
        setSearchText('');
        setRefreshing(false);
    }, []);

    const handleSearch = (text) => {
        setSearchText(text);
        if (text.trim() === '') {
            loadMovies();
        } else {
            try {
                const results = searchMovies(text);
                setMovies(results);
            } catch (error) {
                console.error('Error searching movies:', error);
                setMovies([]);
            }
        }
    };

    const renderMovieItem = ({ item }) => (
        <TouchableOpacity
            style={styles.movieCard}
```


PhucPT@fpt.edu.vn

```
            onPress={() => navigation.navigate('MovieDetail', { movieId: item.id
})}
        >
            <View style={styles.movieHeader}>
                <Text style={styles.movieTitle}>{item.movie_title}</Text>
                <View style={styles.ratingContainer}>
                    <Ionicons name="star" size={16} color="#FFD700" />
                    <Text style={styles.rating}>{item.rating}</Text>
                </View>
            </View>
            <View style={styles.movieInfo}>
                <View style={styles.genreContainer}>
                    <Ionicons name="film-outline" size={14} color="#666" />
                    <Text style={styles.genre}>{item.genre}</Text>
                </View>
                <View style={styles.yearContainer}>
                    <Ionicons name="calendar-outline" size={14} color="#666" />
                    <Text style={styles.year}>{item.release_year}</Text>
                </View>
            </View>
        </TouchableOpacity>
    );

    return (
        <View style={styles.container}>
            <View style={styles.header}>
                <Text style={styles.headerTitle}>Movie Manager</Text>
            </View>

            {/* Banner Image */}
            <Image
                source={require('../../assets/banner.jpg')}
                style={styles.bannerImage}
                resizeMode="cover"
            />

            <View style={styles.searchContainer}>
                <Ionicons name="search" size={20} color="#999"
style={styles.searchIcon} />
                <TextInput
                    style={styles.searchInput}
                    placeholder="Search movies by title..."
                    value={searchText}
                    onChangeText={handleSearch}
                />
                {searchText.length > 0 && (
                    <TouchableOpacity onPress={() => handleSearch('')}>
                        <Ionicons name="close-circle" size={20} color="#999" />
                    </TouchableOpacity>
                )}
```


PhucPT@fpt.edu.vn

```
            </View>

            <FlatList
                data={movies}
                renderItem={renderMovieItem}
                keyExtractor={(item) => item.id.toString()}
                contentContainerStyle={styles.listContainer}
                refreshControl={
                    <RefreshControl refreshing={refreshing}
onRefresh={onRefresh} />
                }
                ListEmptyComponent={
                    <View style={styles.emptyContainer}>
                        <Ionicons name="film-outline" size={64} color="#ccc" />
                        <Text style={styles.emptyText}>No movies found</Text>
                        <Text style={styles.emptySubtext}>
                            {searchText ? 'Try a different search' : 'Add your
first movie!'}
                        </Text>
                    </View>
                }
            />

            {/* FAB - Floating Action Button */}
            <TouchableOpacity
                style={styles.fab}
                onPress={() => navigation.navigate('Add Movie')}
                activeOpacity={0.8}
            >
                <Ionicons name="add" size={28} color="#fff" />
            </TouchableOpacity>
        </View>
    );
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
        backgroundColor: '#f5f5f5',
    },
    header: {
        backgroundColor: '#6200ee',
        paddingTop: 50,
        paddingBottom: 20,
        paddingHorizontal: 20,
    },
    headerTitle: {
        fontSize: 28,
        fontWeight: 'bold',
        color: '#fff',
```


PhucPT@fpt.edu.vn

```
    },
    bannerImage: {
        width: '100%',
        height: 180,
        backgroundColor: '#e0e0e0',
    },
    searchContainer: {
        flexDirection: 'row',
        alignItems: 'center',
        backgroundColor: '#fff',
        margin: 15,
        paddingHorizontal: 15,
        borderRadius: 10,
        elevation: 2,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    searchIcon: {
        marginRight: 10,
    },
    searchInput: {
        flex: 1,
        paddingVertical: 12,
        fontSize: 16,
    },
    listContainer: {
        padding: 15,
        paddingTop: 0,
        paddingBottom: 80, // Extra padding for FAB
    },
    movieCard: {
        backgroundColor: '#fff',
        borderRadius: 12,
        padding: 16,
        marginBottom: 12,
        elevation: 3,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    movieHeader: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        alignItems: 'flex-start',
        marginBottom: 8,
    },
    movieTitle: {
```


PhucPT@fpt.edu.vn

```
        fontSize: 18,
        fontWeight: 'bold',
        color: '#333',
        flex: 1,
        marginRight: 10,
    },
    ratingContainer: {
        flexDirection: 'row',
        alignItems: 'center',
        backgroundColor: '#FFF9E6',
        paddingHorizontal: 8,
        paddingVertical: 4,
        borderRadius: 6,
    },
    rating: {
        fontSize: 16,
        fontWeight: 'bold',
        color: '#333',
        marginLeft: 4,
    },
    movieInfo: {
        flexDirection: 'row',
        alignItems: 'center',
    },
    genreContainer: {
        flexDirection: 'row',
        alignItems: 'center',
        marginRight: 16,
    },
    genre: {
        fontSize: 14,
        color: '#666',
        marginLeft: 4,
    },
    yearContainer: {
        flexDirection: 'row',
        alignItems: 'center',
    },
    year: {
        fontSize: 14,
        color: '#666',
        marginLeft: 4,
    },
    emptyContainer: {
        alignItems: 'center',
        justifyContent: 'center',
        paddingVertical: 60,
    },
    emptyText: {
        fontSize: 18,
```


PhucPT@fpt.edu.vn

```
        fontWeight: 'bold',
        color: '#999',
        marginTop: 16,
    },
    emptySubtext: {
        fontSize: 14,
        color: '#bbb',
        marginTop: 8,
    },
    fab: {
        position: 'absolute',
        right: 20,
        bottom: 20,
        width: 60,
        height: 60,
        borderRadius: 30,
        backgroundColor: '#6200ee',
        justifyContent: 'center',
        alignItems: 'center',
        elevation: 8,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 4 },
        shadowOpacity: 0.3,
        shadowRadius: 8,
    },
});
```

Giải thích:

- `navigation.addListener('focus', loadMovies)` để reload khi quay lại màn hình.
- `toggleWatched` cập nhật cột `watched`.
- Buttons: Add/Edit/Delete.

6) `AddMovieScreen.js` — Create (thêm phim)

`tạo form đơn giản:`

`screens/AddMovieScreen.js:`

```
import React, { useState } from 'react';
import {
    View,
    Text,
    TextInput,
    StyleSheet,
```


PhucPT@fpt.edu.vn

```
    TouchableOpacity,
    ScrollView,
    Alert,
    KeyboardAvoidingView,
    Platform
} from 'react-native';
import { addMovie } from '../database/database';
import { Ionicons } from '@expo/vector-icons';

export default function AddMovieScreen({ navigation }) {
    const [movieTitle, setMovieTitle] = useState('');
    const [genre, setGenre] = useState('');
    const [releaseYear, setReleaseYear] = useState('');
    const [rating, setRating] = useState('');

    const handleSave = () => {
        // Validation
        if (!movieTitle.trim()) {
            Alert.alert('Error', 'Please enter movie title');
            return;
        }
        if (!genre.trim()) {
            Alert.alert('Error', 'Please enter genre');
            return;
        }
        if (!releaseYear.trim() || isNaN(releaseYear)) {
            Alert.alert('Error', 'Please enter valid release year');
            return;
        }
        if (!rating.trim() || isNaN(rating) || parseFloat(rating) < 0 ||
parseFloat(rating) > 5) {
            Alert.alert('Error', 'Please enter valid rating (0.0 - 5.0)');
            return;
        }

        const result = addMovie(
            movieTitle.trim(),
            genre.trim(),
            parseInt(releaseYear),
            parseFloat(rating)
        );

        if (result) {
            Alert.alert('Success', 'Movie added successfully!', [
                {
                    text: 'OK',
                    onPress: () => navigation.goBack()
                }
            ]);
        } else {
```


PhucPT@fpt.edu.vn

```
            Alert.alert('Error', 'Failed to add movie');
        }
    };

    return (
        <KeyboardAvoidingView
            style={styles.container}
            behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
        >
            <ScrollView contentContainerStyle={styles.scrollContainer}>
                <View style={styles.header}>
                    <Ionicons name="add-circle" size={48} color="#6200ee" />
                    <Text style={styles.headerTitle}>Add New Movie</Text>
                </View>

                <View style={styles.form}>
                    <View style={styles.inputGroup}>
                        <Text style={styles.label}>Movie Title *</Text>
                        <View style={styles.inputContainer}>
                            <Ionicons name="film" size={20} color="#6200ee"
style={styles.inputIcon} />
                            <TextInput
                                style={styles.input}
                                placeholder="Enter movie title"
                                value={movieTitle}
                                onChangeText={setMovieTitle}
                            />
                        </View>
                    </View>

                    <View style={styles.inputGroup}>
                        <Text style={styles.label}>Genre *</Text>
                        <View style={styles.inputContainer}>
                            <Ionicons name="pricetag" size={20} color="#6200ee"
style={styles.inputIcon} />
                            <TextInput
                                style={styles.input}
                                placeholder="e.g., Sci-Fi, Action, Drama"
                                value={genre}
                                onChangeText={setGenre}
                            />
                        </View>
                    </View>

                    <View style={styles.inputGroup}>
                        <Text style={styles.label}>Release Year *</Text>
                        <View style={styles.inputContainer}>
                            <Ionicons name="calendar" size={20} color="#6200ee"
style={styles.inputIcon} />
                            <TextInput
```


PhucPT@fpt.edu.vn

```
                                style={styles.input}
                                placeholder="e.g., 2024"
                                value={releaseYear}
                                onChangeText={setReleaseYear}
                                keyboardType="numeric"
                            />
                        </View>
                    </View>

                    <View style={styles.inputGroup}>
                        <Text style={styles.label}>Rating (0.0 - 5.0) *</Text>
                        <View style={styles.inputContainer}>
                            <Ionicons name="star" size={20} color="#6200ee"
style={styles.inputIcon} />
                            <TextInput
                                style={styles.input}
                                placeholder="e.g., 4.5"
                                value={rating}
                                onChangeText={setRating}
                                keyboardType="decimal-pad"
                            />
                        </View>
                    </View>

                    <View style={styles.buttonContainer}>
                        <TouchableOpacity
                            style={styles.cancelButton}
                            onPress={() => navigation.goBack()}
                        >
                            <Ionicons name="close-circle" size={24} color="#666"
/>
                            <Text style={styles.cancelButtonText}>Cancel</Text>
                        </TouchableOpacity>

                        <TouchableOpacity style={styles.saveButton}
onPress={handleSave}>
                            <Ionicons name="checkmark-circle" size={24}
color="#fff" />
                            <Text style={styles.saveButtonText}>Save</Text>
                        </TouchableOpacity>
                    </View>
                </View>
            </ScrollView>
        </KeyboardAvoidingView>
    );
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
```


PhucPT@fpt.edu.vn

```
        backgroundColor: '#f5f5f5',
    },
    scrollContainer: {
        padding: 20,
    },
    header: {
        alignItems: 'center',
        marginBottom: 30,
        marginTop: 20,
    },
    headerTitle: {
        fontSize: 24,
        fontWeight: 'bold',
        color: '#333',
        marginTop: 10,
    },
    form: {
        backgroundColor: '#fff',
        borderRadius: 12,
        padding: 20,
        elevation: 3,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    inputGroup: {
        marginBottom: 20,
    },
    label: {
        fontSize: 16,
        fontWeight: '600',
        color: '#333',
        marginBottom: 8,
    },
    inputContainer: {
        flexDirection: 'row',
        alignItems: 'center',
        borderWidth: 1,
        borderColor: '#ddd',
        borderRadius: 8,
        paddingHorizontal: 12,
        backgroundColor: '#fafafa',
    },
    inputIcon: {
        marginRight: 10,
    },
    input: {
        flex: 1,
        paddingVertical: 12,
```


PhucPT@fpt.edu.vn

```
        fontSize: 16,
        color: '#333',
    },
    buttonContainer: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        marginTop: 10,
        gap: 12,
    },
    cancelButton: {
        flex: 1,
        backgroundColor: '#f5f5f5',
        flexDirection: 'row',
        alignItems: 'center',
        justifyContent: 'center',
        paddingVertical: 15,
        borderRadius: 8,
        borderWidth: 1,
        borderColor: '#ddd',
    },
    cancelButtonText: {
        color: '#666',
        fontSize: 18,
        fontWeight: 'bold',
        marginLeft: 8,
    },
    saveButton: {
        flex: 1,
        backgroundColor: '#6200ee',
        flexDirection: 'row',
        alignItems: 'center',
        justifyContent: 'center',
        paddingVertical: 15,
        borderRadius: 8,
        elevation: 2,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.2,
        shadowRadius: 4,
    },
    saveButtonText: {
        color: '#fff',
        fontSize: 18,
        fontWeight: 'bold',
        marginLeft: 8,
    },
});
```


PhucPT@fpt.edu.vn

7) `EditMovieScreen.js` — Update

screens/EditMovieScreen.js:

```
import React, { useState, useEffect } from 'react';
import {
    View,
    Text,
    TextInput,
    StyleSheet,
    TouchableOpacity,
    ScrollView,
    Alert,
    KeyboardAvoidingView,
    Platform
} from 'react-native';
import { updateMovie } from '../database/database';
import { Ionicons } from '@expo/vector-icons';

export default function EditMovieScreen({ route, navigation }) {
    const { movieId, movie } = route.params;
    const [movieTitle, setMovieTitle] = useState(movie.movie_title);
    const [genre, setGenre] = useState(movie.genre);
    const [releaseYear, setReleaseYear] =
useState(movie.release_year.toString());
    const [rating, setRating] = useState(movie.rating.toString());

    const handleSave = () => {
        // Validation
        if (!movieTitle.trim()) {
            Alert.alert('Error', 'Please enter movie title');
            return;
        }
        if (!genre.trim()) {
            Alert.alert('Error', 'Please enter genre');
            return;
        }
        if (!releaseYear.trim() || isNaN(releaseYear)) {
            Alert.alert('Error', 'Please enter valid release year');
            return;
        }
        if (!rating.trim() || isNaN(rating) || parseFloat(rating) < 0 ||
parseFloat(rating) > 5) {
            Alert.alert('Error', 'Please enter valid rating (0.0 - 5.0)');
            return;
        }

        const success = updateMovie(
            movieId,
            movieTitle.trim(),
```

```
            genre.trim(),
            parseInt(releaseYear),
            parseFloat(rating)
        );

        if (success) {
            Alert.alert('Success', 'Movie updated successfully!', [
                {
                    text: 'OK',
                    onPress: () => navigation.navigate('MovieDetail', { movieId
})
                }
            ]);
        } else {
            Alert.alert('Error', 'Failed to update movie');
        }
    };

    return (
        <KeyboardAvoidingView
            style={styles.container}
            behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
        >
            <ScrollView contentContainerStyle={styles.scrollContainer}>
                <View style={styles.header}>
                    <Ionicons name="create" size={48} color="#2196F3" />
                    <Text style={styles.headerTitle}>Edit Movie</Text>
                </View>

                <View style={styles.form}>
                    <View style={styles.inputGroup}>
                        <Text style={styles.label}>Movie Title *</Text>
                        <View style={styles.inputContainer}>
                            <Ionicons name="film" size={20} color="#2196F3"
style={styles.inputIcon} />
                            <TextInput
                                style={styles.input}
                                placeholder="Enter movie title"
                                value={movieTitle}
                                onChangeText={setMovieTitle}
                            />
                        </View>
                    </View>

                    <View style={styles.inputGroup}>
                        <Text style={styles.label}>Genre *</Text>
                        <View style={styles.inputContainer}>
                            <Ionicons name="pricetag" size={20} color="#2196F3"
style={styles.inputIcon} />
                            <TextInput
```


PhucPT@fpt.edu.vn

```
                            style={styles.input}
                            placeholder="e.g., Sci-Fi, Action, Drama"
                            value={genre}
                            onChangeText={setGenre}
                        />
                    </View>
                </View>

                <View style={styles.inputGroup}>
                    <Text style={styles.label}>Release Year *</Text>
                    <View style={styles.inputContainer}>
                        <Ionicons name="calendar" size={20} color="#2196F3"
style={styles.inputIcon} />
                        <TextInput
                            style={styles.input}
                            placeholder="e.g., 2024"
                            value={releaseYear}
                            onChangeText={setReleaseYear}
                            keyboardType="numeric"
                        />
                    </View>
                </View>

                <View style={styles.inputGroup}>
                    <Text style={styles.label}>Rating (0.0 - 5.0) *</Text>
                    <View style={styles.inputContainer}>
                        <Ionicons name="star" size={20} color="#2196F3"
style={styles.inputIcon} />
                        <TextInput
                            style={styles.input}
                            placeholder="e.g., 4.5"
                            value={rating}
                            onChangeText={setRating}
                            keyboardType="decimal-pad"
                        />
                    </View>
                </View>

                <View style={styles.buttonContainer}>
                    <TouchableOpacity
                        style={styles.cancelButton}
                        onPress={() => navigation.goBack()}
                    >
                        <Ionicons name="close-circle" size={24} color="#666"
/>
                        <Text style={styles.cancelButtonText}>Cancel</Text>
                    </TouchableOpacity>

                    <TouchableOpacity style={styles.saveButton}
onPress={handleSave}>
```


PhucPT@fpt.edu.vn

```
                            <Ionicons name="checkmark-circle" size={24}
color="#fff" />
                            <Text style={styles.saveButtonText}>Save</Text>
                        </TouchableOpacity>
                    </View>
                </View>
            </ScrollView>
        </KeyboardAvoidingView>
    );
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
        backgroundColor: '#f5f5f5',
    },
    scrollContainer: {
        padding: 20,
    },
    header: {
        alignItems: 'center',
        marginBottom: 30,
        marginTop: 20,
    },
    headerTitle: {
        fontSize: 24,
        fontWeight: 'bold',
        color: '#333',
        marginTop: 10,
    },
    form: {
        backgroundColor: '#fff',
        borderRadius: 12,
        padding: 20,
        elevation: 3,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    inputGroup: {
        marginBottom: 20,
    },
    label: {
        fontSize: 16,
        fontWeight: '600',
        color: '#333',
        marginBottom: 8,
    },
    inputContainer: {
```


PhucPT@fpt.edu.vn

```
        flexDirection: 'row',
        alignItems: 'center',
        borderWidth: 1,
        borderColor: '#ddd',
        borderRadius: 8,
        paddingHorizontal: 12,
        backgroundColor: '#fafafa',
    },
    inputIcon: {
        marginRight: 10,
    },
    input: {
        flex: 1,
        paddingVertical: 12,
        fontSize: 16,
        color: '#333',
    },
    buttonContainer: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        marginTop: 10,
        gap: 12,
    },
    cancelButton: {
        flex: 1,
        backgroundColor: '#f5f5f5',
        flexDirection: 'row',
        alignItems: 'center',
        justifyContent: 'center',
        paddingVertical: 15,
        borderRadius: 8,
        borderWidth: 1,
        borderColor: '#ddd',
    },
    cancelButtonText: {
        color: '#666',
        fontSize: 18,
        fontWeight: 'bold',
        marginLeft: 8,
    },
    saveButton: {
        flex: 1,
        backgroundColor: '#2196F3',
        flexDirection: 'row',
        alignItems: 'center',
        justifyContent: 'center',
        paddingVertical: 15,
        borderRadius: 8,
        elevation: 2,
        shadowColor: '#000',
```

```
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.2,
        shadowRadius: 4,
    },
    saveButtonText: {
        color: '#fff',
        fontSize: 18,
        fontWeight: 'bold',
        marginLeft: 8,
    },
});
```

8) `MovieDetailScreen.js` — xem chi tiết

screens/MovieDetailScreen.js:

```
import React, { useState, useEffect } from 'react';
import {
    View,
    Text,
    StyleSheet,
    ScrollView,
    TouchableOpacity,
    Alert
} from 'react-native';
import { getMovieById, deleteMovie } from '../database/database';
import { Ionicons } from '@expo/vector-icons';

export default function MovieDetailScreen({ route, navigation }) {
    const { movieId } = route.params;
    const [movie, setMovie] = useState(null);

    useEffect(() => {
        loadMovie();
    }, []);

    const loadMovie = () => {
        const movieData = getMovieById(movieId);
        setMovie(movieData);
    };

    const handleDelete = () => {
        Alert.alert(
            'Delete Movie',
            'Are you sure you want to delete this movie?',
            [
                {
```

```
                    text: 'Cancel',
                    style: 'cancel'
                },
                {
                    text: 'Delete',
                    style: 'destructive',
                    onPress: () => {
                        const success = deleteMovie(movieId);
                        if (success) {
                            Alert.alert('Success', 'Movie deleted
successfully!', [
                                {
                                    text: 'OK',
                                    onPress: () => navigation.goBack()
                                }
                            ]);
                        } else {
                            Alert.alert('Error', 'Failed to delete movie');
                        }
                    }
                }
            ]
        );
    };

    const handleEdit = () => {
        navigation.navigate('EditMovie', { movieId, movie });
    };

    if (!movie) {
        return (
            <View style={styles.container}>
                <Text style={styles.errorText}>Movie not found</Text>
            </View>
        );
    }

    return (
        <ScrollView style={styles.container}>
            <View style={styles.header}>
                <Ionicons name="film" size={64} color="#6200ee" />
                <Text style={styles.movieTitle}>{movie.movie_title}</Text>
            </View>

            <View style={styles.content}>
                <View style={styles.infoCard}>
                    <View style={styles.infoRow}>
                        <View style={styles.infoLabel}>
                            <Ionicons name="star" size={24} color="#FFD700" />
                            <Text style={styles.labelText}>Rating</Text>
```

```
                        </View>
                        <Text style={styles.infoValue}>{movie.rating} /
5.0</Text>
                    </View>

                    <View style={styles.divider} />

                    <View style={styles.infoRow}>
                        <View style={styles.infoLabel}>
                            <Ionicons name="pricetag" size={24} color="#6200ee"
/>
                            <Text style={styles.labelText}>Genre</Text>
                        </View>
                        <Text style={styles.infoValue}>{movie.genre}</Text>
                    </View>

                    <View style={styles.divider} />

                    <View style={styles.infoRow}>
                        <View style={styles.infoLabel}>
                            <Ionicons name="calendar" size={24} color="#6200ee"
/>
                            <Text style={styles.labelText}>Release Year</Text>
                        </View>
                        <Text
style={styles.infoValue}>{movie.release_year}</Text>
                    </View>

                    <View style={styles.divider} />

                    <View style={styles.infoRow}>
                        <View style={styles.infoLabel}>
                            <Ionicons name="information-circle" size={24}
color="#6200ee" />
                            <Text style={styles.labelText}>Movie ID</Text>
                        </View>
                        <Text style={styles.infoValue}>#{movie.id}</Text>
                    </View>
                </View>

                <View style={styles.actionButtons}>
                    <TouchableOpacity
                        style={[styles.actionButton, styles.editButton]}
                        onPress={handleEdit}
                    >
                        <Ionicons name="create" size={24} color="#fff" />
                        <Text style={styles.actionButtonText}>Edit</Text>
                    </TouchableOpacity>

                    <TouchableOpacity
```

```
                        style={[styles.actionButton, styles.deleteButton]}
                        onPress={handleDelete}
                    >
                        <Ionicons name="trash" size={24} color="#fff" />
                        <Text style={styles.actionButtonText}>Delete</Text>
                    </TouchableOpacity>
                </View>
            </View>
        </ScrollView>
    );
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
        backgroundColor: '#f5f5f5',
    },
    header: {
        backgroundColor: '#6200ee',
        paddingTop: 40,
        paddingBottom: 30,
        paddingHorizontal: 20,
        alignItems: 'center',
    },
    movieTitle: {
        fontSize: 28,
        fontWeight: 'bold',
        color: '#fff',
        marginTop: 15,
        textAlign: 'center',
    },
    content: {
        padding: 20,
    },
    infoCard: {
        backgroundColor: '#fff',
        borderRadius: 12,
        padding: 20,
        marginBottom: 20,
        elevation: 3,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    infoRow: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        alignItems: 'center',
        paddingVertical: 12,
```

```
    },
    infoLabel: {
        flexDirection: 'row',
        alignItems: 'center',
    },
    labelText: {
        fontSize: 16,
        fontWeight: '600',
        color: '#333',
        marginLeft: 12,
    },
    infoValue: {
        fontSize: 16,
        color: '#666',
        fontWeight: '500',
    },
    divider: {
        height: 1,
        backgroundColor: '#eee',
        marginVertical: 8,
    },
    actionButtons: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        gap: 12,
    },
    actionButton: {
        flex: 1,
        flexDirection: 'row',
        alignItems: 'center',
        justifyContent: 'center',
        paddingVertical: 15,
        borderRadius: 8,
        elevation: 2,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.2,
        shadowRadius: 4,
    },
    editButton: {
        backgroundColor: '#2196F3',
    },
    deleteButton: {
        backgroundColor: '#f44336',
    },
    actionButtonText: {
        color: '#fff',
        fontSize: 16,
        fontWeight: 'bold',
        marginLeft: 8,
```


PhucPT@fpt.edu.vn

```
    },
    errorText: {
        fontSize: 18,
        color: '#999',
        textAlign: 'center',
        marginTop: 50,
    },
});
```

9) `StatisticsScreen.js` — Thống kê đơn giản

screens/StatisticsScreen.js:

```
import React, { useState, useEffect } from 'react';
import {
    View,
    Text,
    StyleSheet,
    ScrollView,
    RefreshControl
} from 'react-native';
import { getStatistics } from '../database/database';
import { Ionicons } from '@expo/vector-icons';

export default function StatisticsScreen({ navigation }) {
    const [stats, setStats] = useState({
        totalMovies: 0,
        averageRating: 0,
        genreStats: []
    });
    const [refreshing, setRefreshing] = useState(false);

    useEffect(() => {
        // Load statistics on initial mount
        loadStatistics();

        const unsubscribe = navigation.addListener('focus', () => {
            loadStatistics();
        });

        return unsubscribe;
    }, [navigation]);

    const loadStatistics = () => {
        try {
            const statistics = getStatistics();
            setStats(statistics);
        } catch (error) {
```


PhucPT@fpt.edu.vn

```
            console.error('Error loading statistics:', error);
            // Retry after a short delay
            setTimeout(() => {
                try {
                    const statistics = getStatistics();
                    setStats(statistics);
                } catch (retryError) {
                    console.error('Retry failed:', retryError);
                }
            }, 100);
        }
    };

    const onRefresh = () => {
        setRefreshing(true);
        loadStatistics();
        setRefreshing(false);
    };

    return (
        <ScrollView
            style={styles.container}
            refreshControl={
                <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
            }
        >
            <View style={styles.header}>
                <Ionicons name="stats-chart" size={48} color="#fff" />
                <Text style={styles.headerTitle}>Statistics</Text>
            </View>

            <View style={styles.content}>
                <View style={styles.statsCard}>
                    <View style={styles.statItem}>
                        <View style={styles.statIconContainer}>
                            <Ionicons name="film" size={32} color="#6200ee" />
                        </View>
                        <View style={styles.statInfo}>
                            <Text style={styles.statLabel}>Total Movies</Text>
                            <Text
style={styles.statValue}>{stats.totalMovies}</Text>
                        </View>
                    </View>
                </View>

                <View style={styles.statsCard}>
                    <View style={styles.statItem}>
                        <View style={styles.statIconContainer}>
                            <Ionicons name="star" size={32} color="#FFD700" />
                        </View>
```

```
                        <View style={styles.statInfo}>
                            <Text style={styles.statLabel}>Average Rating</Text>
                            <Text style={styles.statValue}>{stats.averageRating}
/ 5.0</Text>
                        </View>
                    </View>
                </View>

                <View style={styles.genreCard}>
                    <View style={styles.genreHeader}>
                        <Ionicons name="pricetags" size={24} color="#6200ee" />
                        <Text style={styles.genreHeaderText}>Movies by
Genre</Text>
                    </View>

                    {stats.genreStats.length > 0 ? (
                        stats.genreStats.map((genreStat, index) => (
                            <View key={index} style={styles.genreItem}>
                                <View style={styles.genreInfo}>
                                    <View style={styles.genreDot} />
                                    <Text
style={styles.genreName}>{genreStat.genre}</Text>
                                </View>
                                <View style={styles.genreCountContainer}>
                                    <Text
style={styles.genreCount}>{genreStat.count}</Text>
                                    <Text style={styles.genreCountLabel}>
                                        {genreStat.count === 1 ? 'movie' :
'movies'}
                                    </Text>
                                </View>
                            </View>
                        ))
                    ) : (
                        <View style={styles.emptyState}>
                            <Ionicons name="folder-open-outline" size={48}
color="#ccc" />
                            <Text style={styles.emptyText}>No genre data
available</Text>
                        </View>
                    )}
                </View>
            </View>
        </ScrollView>
    );
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
```


PhucPT@fpt.edu.vn

```
        backgroundColor: '#f5f5f5',
    },
    header: {
        backgroundColor: '#6200ee',
        paddingTop: 50,
        paddingBottom: 30,
        paddingHorizontal: 20,
        alignItems: 'center',
    },
    headerTitle: {
        fontSize: 28,
        fontWeight: 'bold',
        color: '#fff',
        marginTop: 10,
    },
    content: {
        padding: 20,
    },
    statsCard: {
        backgroundColor: '#fff',
        borderRadius: 12,
        padding: 20,
        marginBottom: 15,
        elevation: 3,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    statItem: {
        flexDirection: 'row',
        alignItems: 'center',
    },
    statIconContainer: {
        width: 60,
        height: 60,
        borderRadius: 30,
        backgroundColor: '#f0e7ff',
        alignItems: 'center',
        justifyContent: 'center',
        marginRight: 15,
    },
    statInfo: {
        flex: 1,
    },
    statLabel: {
        fontSize: 14,
        color: '#666',
        marginBottom: 5,
    },
```

```
    statValue: {
        fontSize: 28,
        fontWeight: 'bold',
        color: '#333',
    },
    genreCard: {
        backgroundColor: '#fff',
        borderRadius: 12,
        padding: 20,
        elevation: 3,
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 4,
    },
    genreHeader: {
        flexDirection: 'row',
        alignItems: 'center',
        marginBottom: 20,
        paddingBottom: 15,
        borderBottomWidth: 1,
        borderBottomColor: '#eee',
    },
    genreHeaderText: {
        fontSize: 18,
        fontWeight: 'bold',
        color: '#333',
        marginLeft: 10,
    },
    genreItem: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        alignItems: 'center',
        paddingVertical: 12,
        borderBottomWidth: 1,
        borderBottomColor: '#f5f5f5',
    },
    genreInfo: {
        flexDirection: 'row',
        alignItems: 'center',
    },
    genreDot: {
        width: 8,
        height: 8,
        borderRadius: 4,
        backgroundColor: '#6200ee',
        marginRight: 12,
    },
    genreName: {
        fontSize: 16,
```


PhucPT@fpt.edu.vn

```
        color: '#333',
        fontWeight: '500',
    },
    genreCountContainer: {
        alignItems: 'flex-end',
    },
    genreCount: {
        fontSize: 20,
        fontWeight: 'bold',
        color: '#6200ee',
    },
    genreCountLabel: {
        fontSize: 12,
        color: '#999',
        marginTop: 2,
    },
    emptyState: {
        alignItems: 'center',
        paddingVertical: 40,
    },
    emptyText: {
        fontSize: 16,
        color: '#999',
        marginTop: 10,
    },
});
```

10) `index.js` (entry)

Nếu Expo project, `index.js` đơn giản:

```
import { registerRootComponent } from 'expo';
import App from './App';

// registerRootComponent calls AppRegistry.registerComponent('main', () => App);
// It also ensures that whether you load the app in Expo Go or in a native
build,
// the environment is set up appropriately
registerRootComponent(App);
```


PhucPT@fpt.edu.vn

package.json

```
{
  "name": "moviesqlite",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web"
  },
  "dependencies": {
    "@react-navigation/bottom-tabs": "^7.7.3",
    "@react-navigation/native": "^7.1.19",
    "@react-navigation/native-stack": "^7.6.2",
    "expo": "~54.0.20",
    "expo-sqlite": "^16.0.8",
    "expo-status-bar": "~3.0.8",
    "react": "19.1.0",
    "react-native": "0.81.5",
    "react-native-gesture-handler": "~2.28.0",
    "react-native-reanimated": "^4.1.3",
    "react-native-safe-area-context": "^5.6.2",
    "react-native-screens": "~4.16.0"
  },
  "private": true,
  "devDependencies": {
    "babel-preset-expo": "^54.0.6"
  }
}
```

babel.config.js

```
module.exports = function (api) {
    api.cache(true);
    return {
        presets: ['babel-preset-expo'],
        plugins: ['react-native-reanimated/plugin'],
    };
};
```


PhucPT@fpt.edu.vn